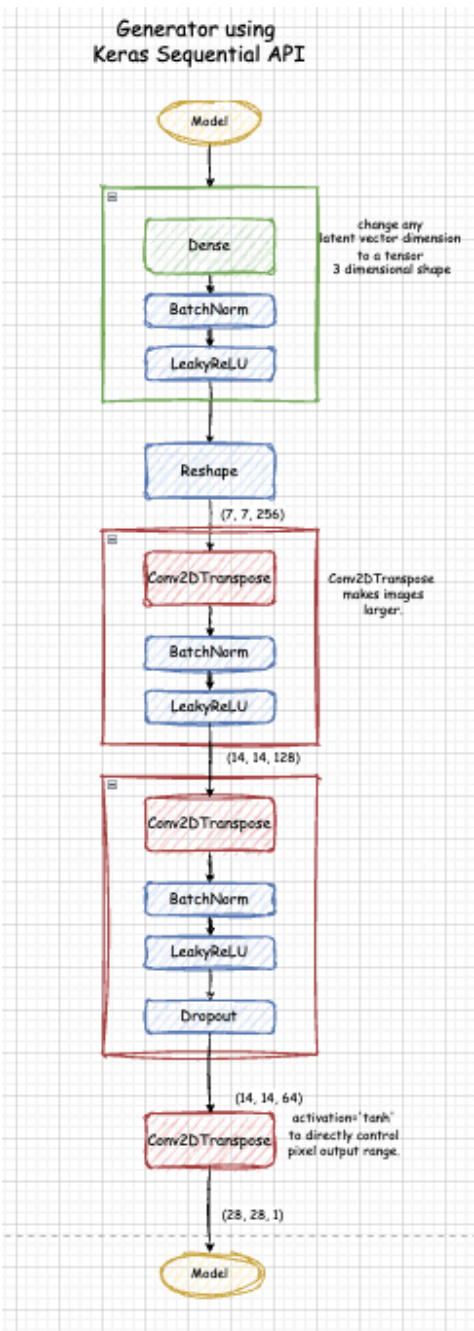
Assignment 3

Generative Adversarial Models

Huong TA

Part 2: Justification



- **LATENT_DIM = 100** -> This is a standard value and works well for most datasets, like MNIST
- **EPOCHS** = 30
- **BATCH_SIZE = 64** -> 64 fits in most GPUs. It's large enough to provide gradient stability

1. **Build the generator model using Keras Sequential API**

- **In the first layer:**

The input shape is (LATENT_DIM,) which is a 1D vector.
LATENT_DIM = 100 => input (100,)
Dense is the first step in the upsampling process, change any latent vector dimension to a tensor 3 dimensional shape.
The first layer will output a tensor of shape (7, 7, 256) which will be upsampled to (28, 28, 1).
The output shape will be (7, 7, 256) after reshaping. 256 is the input channels.

BatchNormalization is a layer in Keras that normalizes the activations of the previous layer per mini-batch. It helps the network train faster and generalize better.

Use LeakyReLU() in a GAN Generator (LeakyReLU stands for Leaky Rectified Linear Unit):

- Prevent dead neurons.
- Maintain gradient flow even with negative values.
- Stabilize training.

Activation that avoids zero-gradient.
By default, alpha=0.3

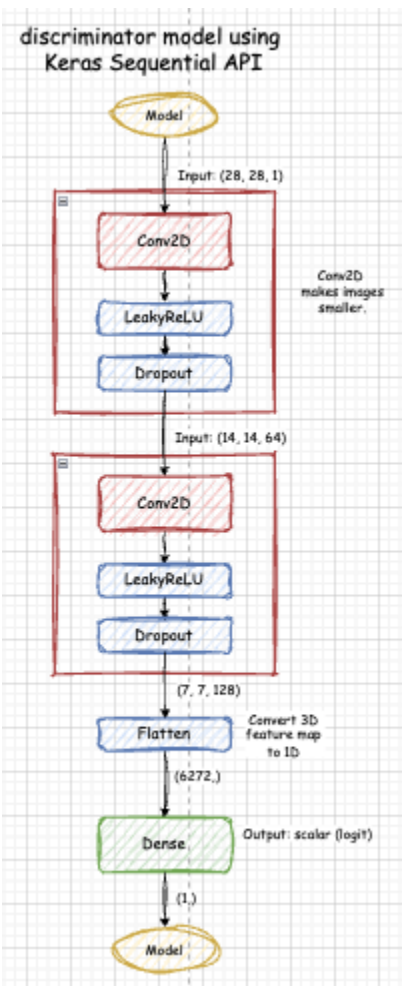
Figure 1: Generator model

- **The second layer: Conv2DTranspose makes images larger.**
    + 128 is the number of filters (output channels).
    + 5x5 is the filter size applied to each input region.
    + Input Size (H, W) and Kernel (Kh, Kw).
    + padding = floor(kernel_size / 2) = 2.

Output size = (Input size−1)×stride+kernel size−2×padding = (7−1)×1+5−2×2 =7 *(because stride = 1, padding = 'same')*

Similar with other Conv2DTranspose layer, do not need BatchNormalization, LeakyReLU and dropout at the last layer

2. **Build the discriminator model using Keras Sequential API**



- The first layer: Conv2D makes images smaller. Add non-linearity and regularization as LeakyReLU and dropout. Get the output shape: (14, 14, 64)
- Similar with the second layer: Conv2D, output shape: (7, 7, 128)
- Flatten(): Convert 3D feature map to 1D
- Dense(1): Output real/fake score (logit)

Dense implements the operation: output = activation(dot(input, kernel) + bias) where activation is the element-wise activation function passed as the activation argument, kernel is a weights matrix created by the layer, and bias is a bias vector created by the layer (only applicable if use_bias is True). These are all attributes of Dense.

Sigmoid makes the output compressed. Very small gradient if output is near 0 or 1 → vanishing gradient

=> Do not use sigmoid in the output of discriminator to training stabilization and clearer gradients: `model.add(layers.Dense(1))`

Figure 2: discriminator

Continue update parameters based on the experiment

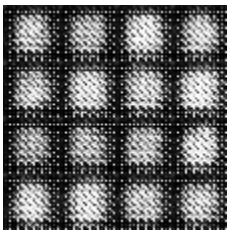
Experiment 01

Losses: The purpose of loss functions is to compute the quantity that a model should seek to minimise during training.

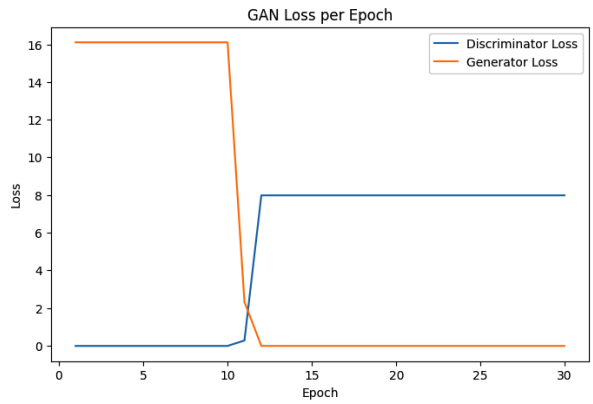
BinaryCrossentropy class: Computes the cross-entropy loss between true labels and predicted labels.

```
loss_fn=keras.losses.BinaryCrossentropy(from_logits=False)
```

Epoch_001



Epoch_030



- Discriminator loss jumps and stabilizes around 5.5
- Generator loss collapses to near 0 ⟶ this means mode collapse or vanishing gradients.

Experiment 02

At the first experiment, outputting raw logits from the discriminator, not probabilities
=> apply from_logits = True to request Keras applies sigmoid inside before calculating loss.
Update:

```
loss_fn=keras.losses.BinaryCrossentropy(from_logits=True)
```

The result:

- Image was improved at the first epochs
- Generator loss (orange): fluctuating around ~1.0
- Discriminator loss (blue): stable, around ~0.5–0.7

Experiment 03

Before:

```
def build_generator():
        model.add(layers.LeakyReLU())
```

After

Add a dropout to help avoid overfitting on the generator's internal layers. Place it after LeakyReLU in deeper layers (after the second Conv2DTranspose). It's more effective after deeper layers, where the generator starts producing mid-level features and has a higher risk of overfitting or overconfidence.

Use LeakyReLU(0.2) instead of the default to improve learning and avoid vanishing gradients.

```
def build_generator():
    model.add(layers.LeakyReLU(alpha=0.2)))
    model.add(layers.Dropout(0.3))
```

The result:

- Discriminator learning curve is more stable
- Generator loss dynamics

=> Keep this update

Experiment 04

EPOCHS = 30

Change Generator Architecture, Increasing filter counts to give generator more capacity (From 256 to 512):

```
model.add(layers.Dense(7 * 7 * 512, use_bias=False))  # More expressive power
...
model.add(layers.Conv2DTranspose(256, ...))
model.add(layers.Conv2DTranspose(128, ...))
model.add(layers.Conv2DTranspose(64, ...))
model.add(layers.Conv2DTranspose(1, ...))
```

The result:

- The Generator loss continues to increase
- Visual quality alone is not sufficient.
- Increasing generator capacity needs longer training or balancing adjustments to show the impact.

=> Try this for larger EPOCHS

Experiment 05

EPOCHS = 50

Generator loss tends to increase more over time in the 50-epoch training, suggesting possible instability or that the discriminator has gained an edge, making it harder for the generator to catch up.
=> Training for longer (50 epochs) does not necessarily improve generator performance — it may even lead to instability.

=> Do not need more Conv2DTranspose layer in Generator

Experiment 06

EPOCHS = 30

After: Add 3 loops for train discriminator

- Make the Discriminator Stronger => gives better feedback (gradients) to the generator.
- Add a Loop for the Discriminator Only (Not Generator) because the Generator Relies on Discriminator Feedback

The result:

- Training the discriminator more frequently helped it learn more effectively and consistently. Its loss stayed lower and showed better convergence.
- The generator struggled more, with loss increasing over time.

This is a classic GAN training imbalance issue: if one model overpowers the other (usually the discriminator), it becomes difficult for the other to learn effectively.

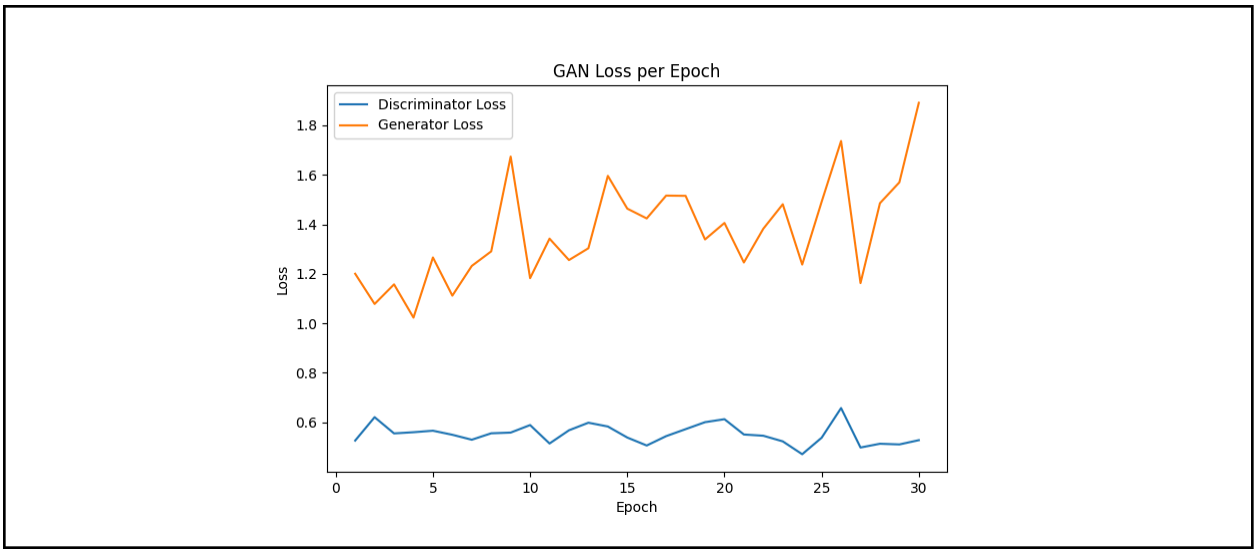
Experiment 07

EPOCHS = 30

=> Reduce to 2 loops for train discriminator => Get the final result.:

- Prevents the discriminator from learning too fast.
- Allows generator to "catch up" in learning.
- Keeps the game between generator and discriminator competitive, not one-sided.

Epoch_001	Epoch_030



Part 3: The training loop and loss function

Analyse the training loop and explain how it works.

Main module: In the main module (Figure 3), the generator and discriminator will be combine into a custom GAN class.

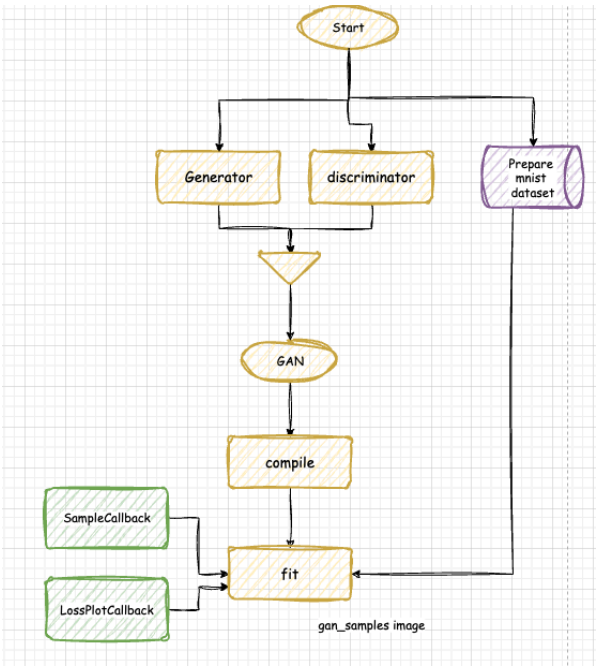


Figure 3: Main function

The GAN model is compiled with appropriate optimizers and a loss function. Training via the fit method, two callbacks are SampleCallBack (to generate and save sample images) and

LossPlotCallBack (to visualize training loss over epochs). The system loops through this process to refine the generator, producing progressively better gan_sample images over time.

In figure 4 is the structure detail, the operation of GAN model. In default, there is only 1 training for discriminator and 1 training for generator. Training start by separate real images to each batch. Generator random latent vector in each time of training.

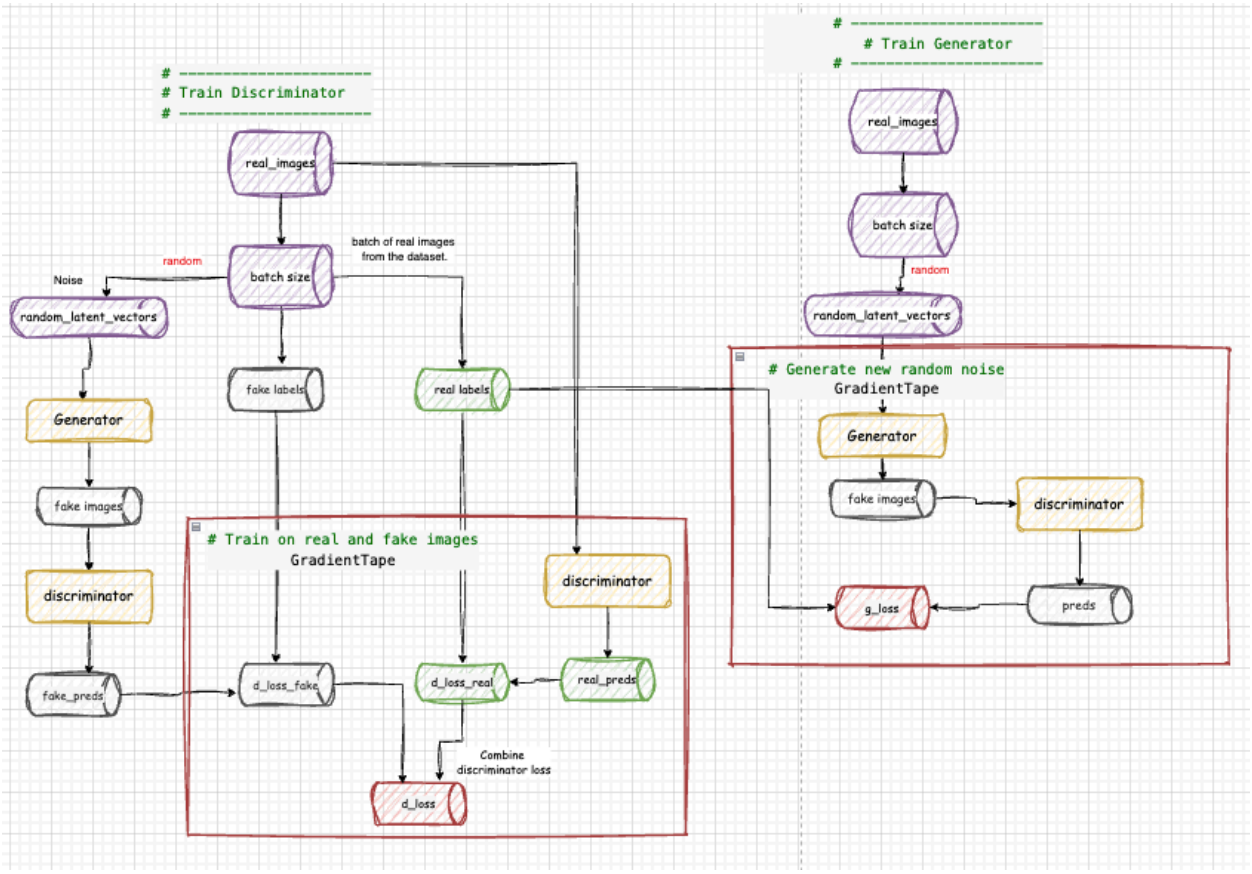


Figure 4: GAN model

Generator will generate fake image from each random latent vector (already add noise to each batch). Disciminator will perform to predict for input images, the input may real or fake.

Explain how the loss function works.

- For the discriminator, it receive two inputs: real images (directly from the database) and the fake images, which was generated with random noise. Perform predictions for all images to get real_preds and fake_preds. The discriminator's loss is computed using a standard loss function:
    + real_preds / real_labels
    + fake_preds / fake_labels

  The final **d_loss** is the average of d_loss_real and d_loss_fake

- For the generator, after generate new set of random noise to get fake images, these will be passed through the discriminator (tried to predict them) => get **g_loss** by compute the predict from discriminator and real images.

Explain, how you could improve the model given additional resources.

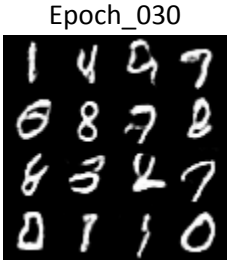
- if I can try with other models: Basic GAN Architecture (DCGAN), StyleGAN, Cycle GAN, BigGAN… After the get loss function from all models, can get the evaluation, which model is best match with the purpose or dataset.
- Data: Try on higher revolution images, add data augmentation to view the final result.
- The better results through experiment, I will try with more parameters in dropout, or LeakyReLU… and in compile also.

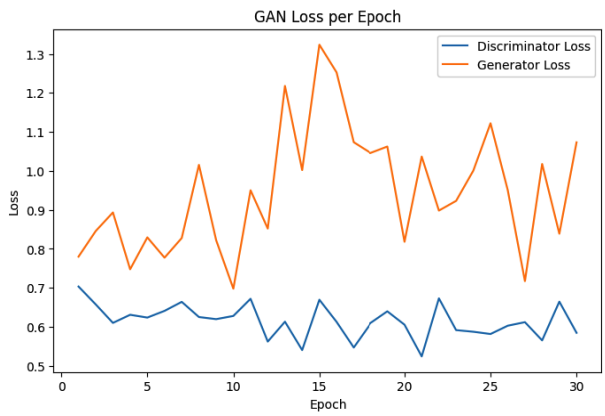
Experiment

Experiment 02

Epoch_001

Epoch_030



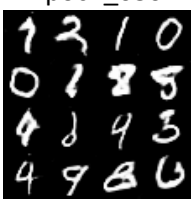


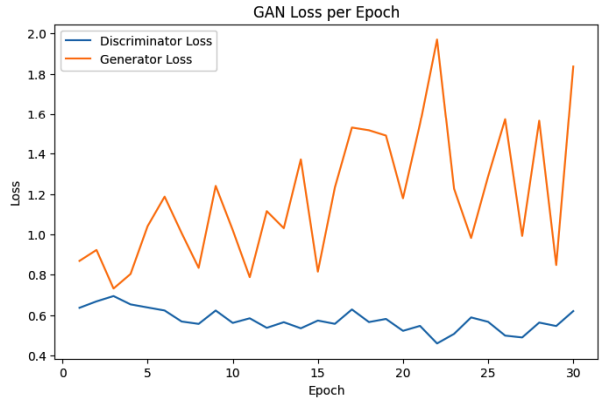
- Image was improved at the first epochs
- Generator loss (orange): fluctuating around ~1.0
- Discriminator loss (blue): stable, around ~0.5–0.7

Experiment 03

Epoch_001

Epoch_030



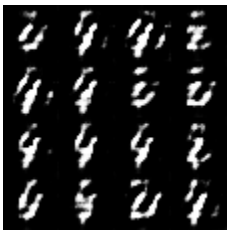


- Discriminator learning curve is more stable
- Generator loss dynamics

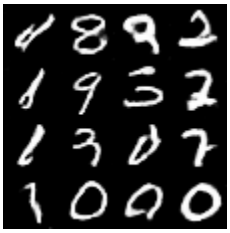
Experiment 04

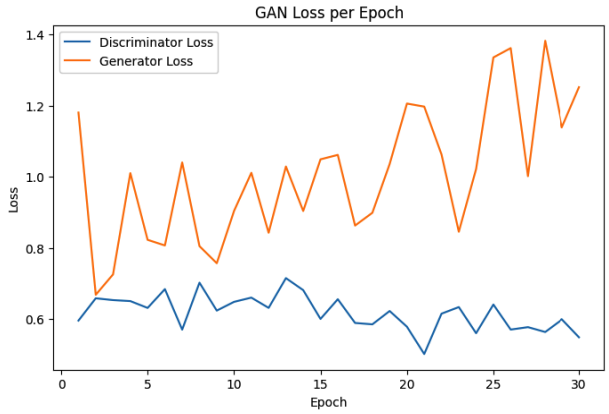
-

Epoch_001



Epoch_030



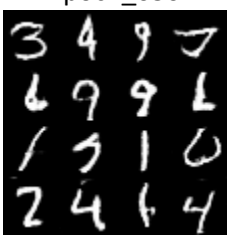


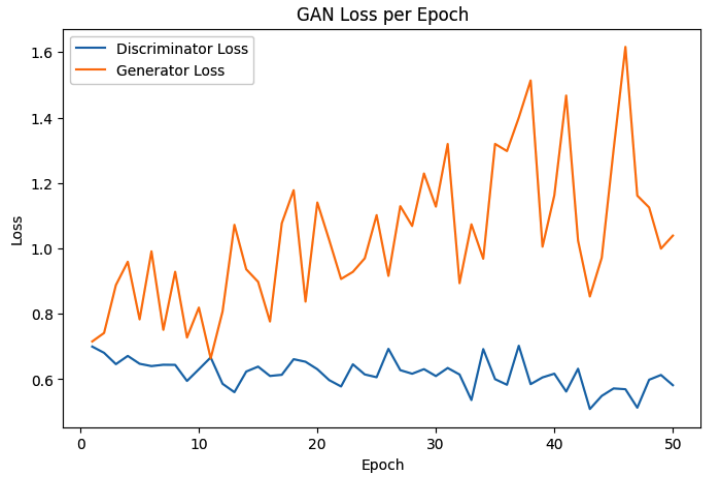
- The Generator loss continuesto increase
- Visual quality alone is not sufficient.
- Increasing generator capacity needs longer training or balancing adjustments to show the impact.

Experiment 05

Epoch_001

Epoch_050

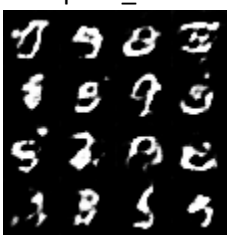




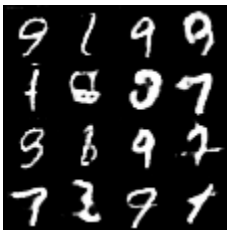
Generator loss tends to increase more over time in the 50-epoch training, suggesting possible instability or that the discriminator has gained an edge, making it harder for the generator to catch up.

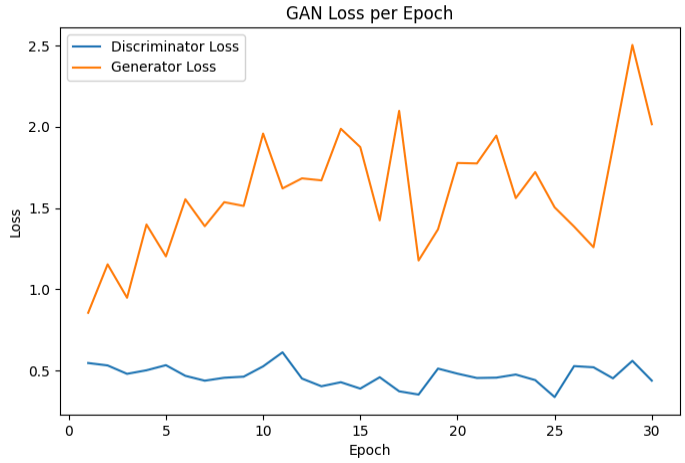
Experiment 06

Epoch_001



Epoch_030





- Training the discriminator more frequently helped it learn more effectively and consistently. Its loss stayed lower and showed better convergence.
- The generator struggled more, with loss increasing over time.